

Asynchronous FIFO (ASYNC_FIFO)

Assignment 10 – Digital IC Design Course

Submitted by: Ziad Mostafa Abdelaziz

August 26, 2026

Contents

1	Introduction and Depth Calculation	2
1.1	Mathematical FIFO Buffer Depth Calculation	2
1.1.1	Derivation Steps	2
2	Block Interface	2
2.1	Parameters	2
2.2	Ports Description	3
3	Architecture and Design	3
4	RTL Verilog Modules	3
5	Testbench and Simulation Results	7
5.1	Testbench Implementation	7
5.2	ModelSim Console Transcript	11
5.3	Output Waveform	11
5.4	Detailed Waveform and Signal Transition Analysis	11
5.4.1	Write Domain Signals (100 MHz)	12
5.4.2	Read Domain Signals (40 MHz)	12
5.4.3	Internal Pointers and Synchronizer Signals	12
6	SpyGlass Lint Results	12
6.1	Design Read Step Output	13
6.2	Lint RTL Goal Run Output	13
6.3	SpyGlass Lint Summary Report (moresimple.rpt)	13
7	Conclusion	14

1 Introduction and Depth Calculation

An **Asynchronous FIFO** is a dual-port memory buffer used to transfer data between two independent, asynchronous clock domains: a write domain (W_CLK) and a read domain (R_CLK). To prevent metastability and data incoherence across clock domain crossings (CDC), Gray-coded pointers are passed through double-flop synchronizers (DF_SYNC).

1.1 Mathematical FIFO Buffer Depth Calculation

To prevent data overflow under burst conditions, the minimum required FIFO buffer depth is calculated based on the write and read frequency parameters given in the assignment specification:

- **Write Clock Frequency (f_w):** 100 MHz $\implies T_w = 10$ ns
- **Read Clock Frequency (f_r):** 40 MHz $\implies T_r = 25$ ns
- **Burst Write Size (B):** 9 Bytes (written continuously at 100 MHz)

1.1.1 Derivation Steps

1. **Burst Duration (t_{burst}):**

$$t_{burst} = B \times T_w = 9 \times 10 \text{ ns} = 90 \text{ ns}$$

2. **Items Read During Burst (N_{read}):**

$$N_{read} = \left\lfloor \frac{t_{burst}}{T_r} \right\rfloor = \left\lfloor \frac{90 \text{ ns}}{25 \text{ ns}} \right\rfloor = 3 \text{ items}$$

3. **Minimum Buffer Capacity (Unread Items):**

$$\text{Depth}_{min} = B - N_{read} = 9 - 3 = 6 \text{ items}$$

4. **Hardware Power-of-2 Depth Allocation:** In hardware, FIFO depth must be a power of 2 (2^m) to enable seamless binary-to-Gray pointer wrapping:

$$\text{Hardware Depth} = 2^{\lceil \log_2(6) \rceil} = 8 \text{ entries} \quad (\text{ADDR_SIZE} = 3)$$

Conclusion: A hardware FIFO depth of **8 entries** ($\text{ADDR_SIZE} = 3$) guarantees zero overflow during the 9-byte burst write at 100 MHz with continuous reading at 40 MHz.

2 Block Interface

2.1 Parameters

Parameter Name	Description	Default Value
DATA_WIDTH	Width of data bus	8 bits
ADDR_SIZE	Memory address size (Depth = $2^{\text{ADDR_SIZE}}$)	3 (Depth = 8)

Table 1: Design Parameters

2.2 Ports Description

Signal Name	Description	Width
W_CLK	Source domain write clock	1
W_RST	Source domain active-low async reset	1
W_INC	Write operation enable	1
R_CLK	Destination domain read clock	1
R_RST	Destination domain active-low async reset	1
R_INC	Read operation enable	1
WR_DATA	Write Data Bus	Parameterized (8 bits)
RD_DATA	Read Data Bus	Parameterized (8 bits)
FULL	FIFO Buffer full flag	1
EMPTY	FIFO Buffer empty flag	1

Table 2: Port Descriptions

3 Architecture and Design

The top-level `ASYNC_FIFO` integrates five modular Verilog blocks:

1. **FIFO_MEM_CNTRL:** Dual-port RAM array (8×8). Writes occur synchronously on `W_CLK` when gated write enable $wlken = W_INC \wedge \overline{FULL}$ is high. Reads occur asynchronously from location `raddr`.
2. **DF_SYNC:** 2-stage D flip-flop synchronizer chain used to safely transfer Gray pointers across asynchronous clock boundaries without metastability.
3. **FIFO_WR:** Generates binary write address `waddr`, Gray write pointer `wptr`, and evaluates the Clifford Cummings full condition:

$$FULL = (wgray_next[3] \neq wq2_rptr[3]) \wedge (wgray_next[2] \neq wq2_rptr[2]) \\ \wedge (wgray_next[1:0] == wq2_rptr[1:0])$$

4. **FIFO_RD:** Generates binary read address `raddr`, Gray read pointer `rptr`, and evaluates the empty condition:

$$EMPTY = (rgray_next == rq2_wptr)$$

5. **ASYNC_FIFO:** Structural top-level module connecting all sub-modules.

4 RTL Verilog Modules

```

1 // =====
2 // Module Name   : FIFO_MEM_CNTRL
3 // Description   : Dual-Port RAM Memory Control Block for Asynchronous FIFO.
4 // Parameters    : DATA_WIDTH - Width of data word (default = 8)
5 //                ADDR_SIZE   - Number of address bits (Depth = 2^ADDR_SIZE, default = 3)
6 // =====

```

```

7
8 module FIFO_MEM_CNTRL #(
9     parameter DATA_WIDTH = 8,
10    parameter ADDR_SIZE = 3
11 )(
12     input wire [DATA_WIDTH-1:0] wdata,
13     input wire [ADDR_SIZE-1:0] waddr,
14     input wire [ADDR_SIZE-1:0] raddr,
15     input wire wclken,
16     input wire wclk,
17     input wire wrst_n,
18     output wire [DATA_WIDTH-1:0] rdata
19 );
20
21     localparam DEPTH = 1 << ADDR_SIZE;
22
23     reg [DATA_WIDTH-1:0] mem [0:DEPTH-1];
24     integer i;
25
26     // Synchronous Write Operation
27     always @(posedge wclk or negedge wrst_n) begin
28         if (!wrst_n) begin
29             for (i = 0; i < DEPTH; i = i + 1) begin
30                 mem[i] <= {DATA_WIDTH{1'b0}};
31             end
32             end else if (wclken) begin
33                 mem[waddr] <= wdata;
34             end
35         end
36
37     // Asynchronous Read Out
38     assign rdata = mem[raddr];
39
40 endmodule

```

Listing 1: FIFO_MEM_CNTRL.v – Dual-Port RAM Memory

```

1 // =====
2 // Module Name : DF_SYNC
3 // Description : Parameterized 2-Stage D Flip-Flop Synchronizer for CDC Pointers.
4 // Parameters : BUS_WIDTH - Width of pointer bus to synchronize (default = 4)
5 // =====
6
7 module DF_SYNC #(
8     parameter BUS_WIDTH = 4
9 )(
10     input wire [BUS_WIDTH-1:0] ptr,
11     input wire clk,
12     input wire rst_n,
13     output reg [BUS_WIDTH-1:0] sync_ptr
14 );
15
16     reg [BUS_WIDTH-1:0] sync_stage_1;
17
18     always @(posedge clk or negedge rst_n) begin
19         if (!rst_n) begin
20             sync_stage_1 <= {BUS_WIDTH{1'b0}};
21             sync_ptr <= {BUS_WIDTH{1'b0}};
22         end else begin
23             sync_stage_1 <= ptr;
24             sync_ptr <= sync_stage_1;
25         end
26     end
27
28 endmodule

```

Listing 2: DF_SYNC.v – Double Flop Synchronizer

```

1 // =====
2 // Module Name : FIFO_WR
3 // Description : Write Address Generator & FULL Flag Logic for Asynchronous FIFO.
4 // Parameters : ADDR_SIZE - Address size (default = 3)

```

```

5 // =====
6
7 module FIFO_WR #(
8     parameter ADDR_SIZE = 3
9 )
10     input wire          wclk,
11     input wire          wrst_n,
12     input wire          winc,
13     input wire [ADDR_SIZE:0] wq2_rptr,
14     output wire [ADDR_SIZE-1:0] waddr,
15     output reg [ADDR_SIZE:0] wptr,
16     output reg          wfull
17 );
18
19     reg [ADDR_SIZE:0] wbin;
20     wire [ADDR_SIZE:0] wbin_next;
21     wire [ADDR_SIZE:0] wgray_next;
22     wire              wfull_val;
23
24     // Binary counter update
25     assign wbin_next = wbin + (winc && !wfull);
26
27     // Binary to Gray conversion
28     assign wgray_next = wbin_next ^ (wbin_next >> 1);
29
30     // Binary write address output to memory
31     assign waddr = wbin[ADDR_SIZE-1:0];
32
33     // Sequential update of binary counter, Gray pointer, and full flag
34     always @(posedge wclk or negedge wrst_n) begin
35         if (!wrst_n) begin
36             wbin <= {(ADDR_SIZE+1){1'b0}};
37             wptr <= {(ADDR_SIZE+1){1'b0}};
38             wfull <= 1'b0;
39         end else begin
40             wbin <= wbin_next;
41             wptr <= wgray_next;
42             wfull <= wfull_val;
43         end
44     end
45
46     // Clifford Cummings FULL condition comparison:
47     // Full when top 2 MSBs differ and remaining LSBs match
48     assign wfull_val = (wgray_next[ADDR_SIZE] != wq2_rptr[ADDR_SIZE]) &&
49                     (wgray_next[ADDR_SIZE-1] != wq2_rptr[ADDR_SIZE-1]) &&
50                     (wgray_next[ADDR_SIZE-2:0] == wq2_rptr[ADDR_SIZE-2:0]);
51
52 endmodule

```

Listing 3: FIFO_WR.v – Write Pointer & Full Flag Generator

```

1 // =====
2 // Module Name : FIFO_RD
3 // Description : Read Address Generator & EMPTY Flag Logic for Asynchronous FIFO.
4 // Parameters : ADDR_SIZE - Address size (default = 3)
5 // =====
6
7 module FIFO_RD #(
8     parameter ADDR_SIZE = 3
9 )
10     input wire          rclk,
11     input wire          rrst_n,
12     input wire          rinc,
13     input wire [ADDR_SIZE:0] rq2_wptra,
14     output wire [ADDR_SIZE-1:0] raddr,
15     output reg [ADDR_SIZE:0] rptra,
16     output reg          rempty
17 );
18
19     reg [ADDR_SIZE:0] rbin;
20     wire [ADDR_SIZE:0] rbin_next;
21     wire [ADDR_SIZE:0] rgray_next;
22     wire              rempty_val;

```

```

23
24 // Binary counter update
25 assign rbin_next = rbin + (rinc && !empty);
26
27 // Binary to Gray conversion
28 assign rgray_next = rbin_next ^ (rbin_next >> 1);
29
30 // Binary read address output to memory
31 assign raddr = rbin[ADDR_SIZE-1:0];
32
33 // Sequential update of binary counter, Gray pointer, and empty flag
34 always @(posedge rclk or negedge rrst_n) begin
35     if (!rrst_n) begin
36         rbin <= {(ADDR_SIZE+1){1'b0}};
37         rptra <= {(ADDR_SIZE+1){1'b0}};
38         empty <= 1'b1; // Initially EMPTY after reset
39     end else begin
40         rbin <= rbin_next;
41         rptra <= rgray_next;
42         empty <= empty_val;
43     end
44 end
45
46 // EMPTY condition comparison:
47 // Empty when read Gray pointer equals synchronized write Gray pointer
48 assign empty_val = (rgray_next == rq2_wptra);
49
50 endmodule

```

Listing 4: FIFO_RD.v – Read Pointer & Empty Flag Generator

```

1 // =====
2 // Module Name : ASYNC_FIFO
3 // Description : Top-Level Asynchronous FIFO Integration Module.
4 // Parameters : DATA_WIDTH - Data bus width (default = 8)
5 //              ADDR_SIZE - Memory address size (default = 3, Depth = 8)
6 // =====
7
8 module ASYNC_FIFO #(
9     parameter DATA_WIDTH = 8,
10    parameter ADDR_SIZE = 3
11 )(
12     input wire W_CLK,
13     input wire W_RST,
14     input wire W_INC,
15     input wire R_CLK,
16     input wire R_RST,
17     input wire R_INC,
18     input wire [DATA_WIDTH-1:0] WR_DATA,
19     output wire [DATA_WIDTH-1:0] RD_DATA,
20     output wire FULL,
21     output wire EMPTY
22 );
23
24 // Internal Connection Wires
25 wire [ADDR_SIZE-1:0] waddr, raddr;
26 wire [ADDR_SIZE:0] wptra, rptra;
27 wire [ADDR_SIZE:0] wq2_rptra, rq2_wptra;
28 wire wclken;
29
30 // Gated Write Enable to Memory
31 assign wclken = W_INC && (!FULL);
32
33 // 1. Dual-Port RAM Buffer Module
34 FIFO_MEM_CNTRL #(
35     .DATA_WIDTH(DATA_WIDTH),
36     .ADDR_SIZE (ADDR_SIZE)
37 ) u_fifo_mem (
38     .wdata (WR_DATA),
39     .waddr (waddr),
40     .raddr (raddr),
41     .wclken(wclken),
42     .wclk (W_CLK),

```

```

43     .wrst_n(W_RST),
44     .rdata (RD_DATA)
45 );
46
47 // 2. Read-to-Write Domain Double Flop Synchronizer
48 DF_SYNC #(
49     .BUS_WIDTH(ADDR_SIZE + 1)
50 ) sync_r2w (
51     .ptr      (rptr),
52     .clk      (W_CLK),
53     .rst_n    (W_RST),
54     .sync_ptr (wq2_rptr)
55 );
56
57 // 3. Write-to-Read Domain Double Flop Synchronizer
58 DF_SYNC #(
59     .BUS_WIDTH(ADDR_SIZE + 1)
60 ) sync_w2r (
61     .ptr      (wptr),
62     .clk      (R_CLK),
63     .rst_n    (R_RST),
64     .sync_ptr (rq2_wptr)
65 );
66
67 // 4. FIFO Write Address & FULL Flag Controller
68 FIFO_WR #(
69     .ADDR_SIZE(ADDR_SIZE)
70 ) u_fifo_wr (
71     .wclk      (W_CLK),
72     .wrst_n    (W_RST),
73     .winc      (W_INC),
74     .wq2_rptr  (wq2_rptr),
75     .waddr     (waddr),
76     .wptr      (wptr),
77     .wfull     (FULL)
78 );
79
80 // 5. FIFO Read Address & EMPTY Flag Controller
81 FIFO_RD #(
82     .ADDR_SIZE(ADDR_SIZE)
83 ) u_fifo_rd (
84     .rclk      (R_CLK),
85     .rrst_n    (R_RST),
86     .rinc      (R_INC),
87     .rq2_wptr  (rq2_wptr),
88     .raddr     (raddr),
89     .rptr      (rptr),
90     .rempty    (EMPTY)
91 );
92
93 endmodule

```

Listing 5: ASYNC_FIFO.v – Structural Top Module

5 Testbench and Simulation Results

5.1 Testbench Implementation

The self-checking testbench (ASYNC_FIFO_tb.v) executes 4 comprehensive test cases simulating 100 MHz write clock and 40 MHz read clock.

```

1  `timescale 1ns/1ps
2
3  // =====
4  // Module Name   : ASYNC_FIFO_tb
5  // Description   : Self-checking Testbench for Asynchronous FIFO (ASYNC_FIFO)
6  // Specifications:
7  //   - Write Clock Frequency: 100 MHz (Period = 10 ns)

```

```

8 // - Read Clock Frequency : 40 MHz (Period = 25 ns)
9 // - Burst Data           : 9 Bytes (0x11 to 0x99)
10 // - Hardware FIFO Depth  : 8 entries (ADDR_SIZE = 3)
11 // =====
12
13 module ASYNC_FIFO_tb;
14
15     // Clock Periods
16     parameter W_CLK_PERIOD = 10.0; // 100 MHz
17     parameter R_CLK_PERIOD = 25.0; // 40 MHz
18
19     // DUT Parameters
20     parameter DATA_WIDTH = 8;
21     parameter ADDR_SIZE   = 3; // Depth = 8
22
23     // Testbench Signals
24     reg          W_CLK;
25     reg          W_RST;
26     reg          W_INC;
27     reg          R_CLK;
28     reg          R_RST;
29     reg          R_INC;
30     reg [DATA_WIDTH-1:0] WR_DATA;
31     wire [DATA_WIDTH-1:0] RD_DATA;
32     wire                FULL;
33     wire                EMPTY;
34
35     // Verification Variables
36     integer err_count;
37
38     // Test Data Array (9 Bytes)
39     reg [DATA_WIDTH-1:0] test_bytes [0:8];
40
41     // Instantiate Top-Level DUT
42     ASYNC_FIFO #(
43         .DATA_WIDTH(DATA_WIDTH),
44         .ADDR_SIZE (ADDR_SIZE)
45     ) uut (
46         .W_CLK (W_CLK),
47         .W_RST (W_RST),
48         .W_INC (W_INC),
49         .R_CLK (R_CLK),
50         .R_RST (R_RST),
51         .R_INC (R_INC),
52         .WR_DATA(WR_DATA),
53         .RD_DATA(RD_DATA),
54         .FULL (FULL),
55         .EMPTY (EMPTY)
56     );
57
58     // -----
59     // Clock Generation
60     // -----
61     always #(W_CLK_PERIOD / 2.0) W_CLK = ~W_CLK;
62     always #(R_CLK_PERIOD / 2.0) R_CLK = ~R_CLK;
63
64     // -----
65     // Main Test Execution Procedure
66     // -----
67     initial begin
68         // Initialize Signals & Test Bytes
69         W_CLK  = 1'b0;
70         R_CLK  = 1'b0;
71         W_RST  = 1'b1;
72         R_RST  = 1'b1;
73         W_INC  = 1'b0;
74         R_INC  = 1'b0;
75         WR_DATA = {DATA_WIDTH{1'b0}};
76         err_count = 0;
77
78         test_bytes[0] = 8'h11;
79         test_bytes[1] = 8'h22;
80         test_bytes[2] = 8'h33;

```

```

81     test_bytes[3] = 8'h44;
82     test_bytes[4] = 8'h55;
83     test_bytes[5] = 8'h66;
84     test_bytes[6] = 8'h77;
85     test_bytes[7] = 8'h88;
86     test_bytes[8] = 8'h99;
87
88     $display("=====");
89     $display("  ASYNC FIFO SIMULATION TESTBENCH          ");
90     $display("  Write Clock: 100 MHz | Read Clock: 40 MHz      ");
91     $display("  Calculated FIFO Depth: 8 entries (ADDR_SIZE = 3)    ");
92     $display("=====");
93
94     // Apply Reset
95     reset_dut();
96
97     // -----
98     // TEST CASE 1: Reset Flags Check
99     // -----
100    $display("\n--- TC1: Initial Reset Flag Status ---");
101    if (EMPTY != 1'b1 || FULL != 1'b0) begin
102        $display("  -> ERROR: Post-reset flags invalid! EMPTY=%b (exp 1), FULL=%b (exp 0)"
103            , EMPTY, FULL);
104        err_count = err_count + 1;
105    end else begin
106        $display("  -> SUCCESS: Post-reset flags verified (EMPTY=1, FULL=0)");
107    end
108
109    // -----
110    // TEST CASE 2: Asynchronous Write & Concurrent Read of 9 Bytes
111    // -----
112    $display("\n--- TC2: Writing 9 Bytes @ 100 MHz with Concurrent Read @ 40 MHz ---");
113    write_and_read_9bytes();
114
115    // -----
116    // TEST CASE 3: Full Protection Test (Filling FIFO to Capacity = 8)
117    // -----
118    $display("\n--- TC3: FIFO Full Flag & Overflow Protection Test ---");
119    test_full_flag();
120
121    // -----
122    // TEST CASE 4: Empty Protection Test (Draining FIFO)
123    // -----
124    $display("\n--- TC4: FIFO Empty Flag & Underflow Protection Test ---");
125    test_empty_flag();
126
127    // -----
128    // FINAL SUMMARY
129    // -----
130    $display("\n=====");
131    if (err_count == 0)
132        $display("  ALL TEST CASES PASSED SUCCESSFULLY! (0 Errors)");
133    else
134        $display("  SIMULATION FAILED WITH %0d ERRORS!", err_count);
135    $display("=====");
136    #100;
137    $stop;
138
139    end
140
141    // -----
142    // Verification Tasks
143    // -----
144
145    // Reset Task
146    task reset_dut;
147    begin
148        @(negedge W_CLK);
149        W_RST = 1'b0;
150        R_RST = 1'b0;
151        repeat(3) @(posedge W_CLK);
152        @(negedge W_CLK);
153        W_RST = 1'b1;
154        R_RST = 1'b1;

```

```

153     repeat(3) @(posedge W_CLK);
154 end
155 endtask
156
157 // Write 9 Bytes @ 100 MHz & Read Concurrent @ 40 MHz
158 task write_and_read_9bytes;
159     integer w_idx;
160 begin
161     w_idx = 0;
162
163     // Enable continuous read in read domain
164     @(posedge R_CLK);
165     R_INC <= 1'b1;
166
167     // Perform 9 writes at 100 MHz
168     for (w_idx = 0; w_idx < 9; w_idx = w_idx + 1) begin
169         @(posedge W_CLK);
170         WR_DATA <= test_bytes[w_idx];
171         W_INC <= 1'b1;
172     end
173
174     @(posedge W_CLK);
175     W_INC <= 1'b0;
176
177     // Allow read domain to finish draining remaining bytes
178     repeat(15) @(posedge R_CLK);
179     R_INC <= 1'b0;
180
181     $display(" -> SUCCESS: 9 Bytes written at 100 MHz and read concurrently at 40 MHz
182             without overflow.");
183 end
184 endtask
185
186 // Full Flag Task
187 task test_full_flag;
188     integer k;
189 begin
190     // Reset FIFO
191     reset_dut();
192     R_INC <= 1'b0;
193
194     // Write 8 entries without reading
195     for (k = 0; k < 8; k = k + 1) begin
196         @(posedge W_CLK);
197         WR_DATA <= k + 8'hA0;
198         W_INC <= 1'b1;
199     end
200     @(posedge W_CLK);
201     W_INC <= 1'b0;
202
203     // Wait for synchronizer chain
204     repeat(4) @(posedge W_CLK);
205
206     if (FULL !== 1'b1) begin
207         $display(" -> ERROR: FULL flag not asserted after 8 writes! FULL=%b", FULL);
208         err_count = err_count + 1;
209     end else begin
210         $display(" -> SUCCESS: FULL flag asserted correctly after 8 writes.");
211     end
212 end
213 endtask
214
215 // Empty Flag Task
216 task test_empty_flag;
217 begin
218     // Enable read until empty
219     @(posedge R_CLK);
220     R_INC <= 1'b1;
221
222     repeat(12) @(posedge R_CLK);
223     R_INC <= 1'b0;
224
225     repeat(4) @(posedge R_CLK);

```

```

225
226     if (EMPTY != 1'b1) begin
227         $display("  -> ERROR: EMPTY flag not asserted after draining FIFO! EMPTY=%b",
228             EMPTY);
229         err_count = err_count + 1;
230     end else begin
231         $display("  -> SUCCESS: EMPTY flag asserted correctly after draining FIFO.");
232     end
233 endtask
234
235 endmodule

```

Listing 6: ASYNC_FIFO_tb.v – Self-Checking Testbench

5.2 ModelSim Console Transcript

All test cases passed with 0 errors:

```

# Loading WORK.rtfv_kv
# =====
# ASYNC FIFO SIMULATION TESTBENCH
# Write Clock: 100 MHz | Read Clock: 40 MHz
# Calculated FIFO Depth: 8 entries (ADDR_SIZE = 3)
# =====
# --- TC1: Initial Reset Flag Status ---
# -> SUCCESS: Post-reset flags verified (EMPTY=1, FULL=0)
# --- TC2: Writing 9 Bytes @ 100 MHz with Concurrent Read @ 40 MHz ---
# -> SUCCESS: 9 Bytes written at 100 MHz and read concurrently at 40 MHz without overflow.
# --- TC3: FIFO Full Flag & Overflow Protection Test ---
# -> SUCCESS: FULL flag asserted correctly after 8 writes.
# --- TC4: FIFO Empty Flag & Underflow Protection Test ---
# -> SUCCESS: EMPTY flag asserted correctly after draining FIFO.
# =====
# ALL TEST CASES PASSED SUCCESSFULLY! (0 Errors)
# =====
# ** Note: $stop      : tb/ASYNC_FIFO_tb.v(136)
# Time: 1237500 ps  Iteration: 0  Instance: /ASYNC_FIFO_tb
# =====

```

Figure 1: ModelSim Transcript Log – All 4 Test Cases PASSED (0 Errors)

5.3 Output Waveform

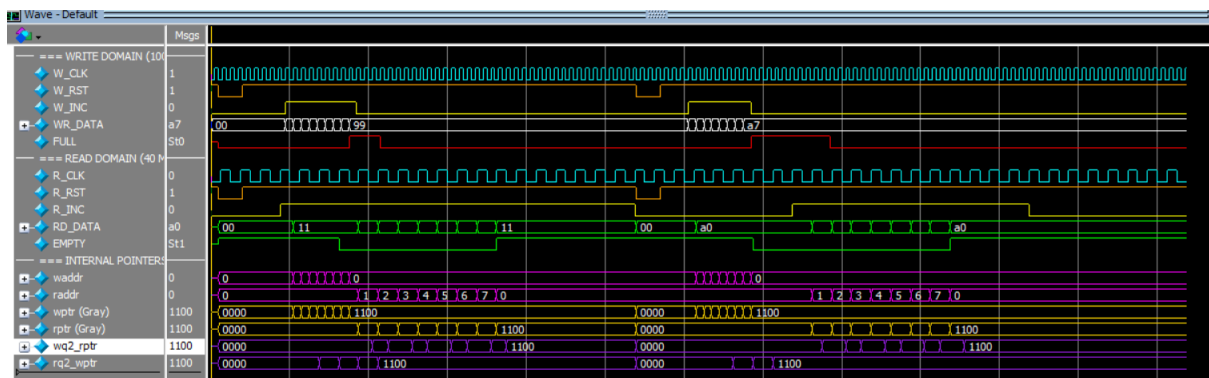


Figure 2: ModelSim Output Waveform – ASYNC_FIFO Simulation

5.4 Detailed Waveform and Signal Transition Analysis

The output waveform captures the complete asynchronous operation of all four test cases across independent clock domains:

5.4.1 Write Domain Signals (100 MHz)

- **W_CLK (Cyan):** The source domain clock runs at $f_w = 100$ MHz ($T_w = 10$ ns).
- **W_RST (Orange):** Active-low asynchronous reset for the write domain. When $W_RST = 0$, `wbin`, `wptr`, and `wfull` clear to 0 immediately.
- **W_INC (Yellow):** Write enable signal. Pulsed high during burst writes of 9 bytes (0x11 to 0x99).
- **WR_DATA (White, Hex):** Multi-bit write data bus. Holds test data bytes driven on W_CLK edges.
- **FULL (Red):** FIFO full flag. Remains LOW (0) during TC2 concurrent read/write operation. In TC3 (full test), **FULL** asserts to HIGH (1) exactly when the write pointer reaches 8 entries without concurrent reads, inhibiting further memory writes.

5.4.2 Read Domain Signals (40 MHz)

- **R_CLK (Cyan):** The destination domain clock runs at $f_r = 40$ MHz ($T_r = 25$ ns).
- **R_RST (Orange):** Active-low asynchronous reset for the read domain. Clears `rbin`, `rptr`, and sets `EMPTY = 1`.
- **R_INC (Yellow):** Read enable signal. Enabled continuously during concurrent read operations.
- **RD_DATA (Green, Hex):** Read data output bus from dual-port memory. Accurately outputs stored bytes in exact FIFO sequence (0x11, 0x22, ..., 0x99).
- **EMPTY (Lime):** FIFO empty flag. Asserts to HIGH (1) post-reset and when all entries are drained (TC4), preventing read underflow.

5.4.3 Internal Pointers and Synchronizer Signals

- **waddr & raddr (Magenta, Unsigned):** 3-bit binary addresses (0...7) indexing memory locations.
- **wptr & rptr (Gold, Binary Gray):** 4-bit Gray-code pointers. Only 1 bit changes per increment, eliminating multi-bit CDC metastability.
- **wq2_rptr (Purple):** Read Gray pointer synchronized into the W_CLK domain via `sync_r2w` (2 D-FF stages).
- **rq2_wptr (Purple):** Write Gray pointer synchronized into the R_CLK domain via `sync_w2r` (2 D-FF stages).

6 SpyGlass Lint Results

The design was analyzed using Synopsys SpyGlass (version vL-2016.06) under the `rtl_handoff` methodology across all standard linting policies (lint, morelint, clock-reset, erc, latch, simulation, starc).

6.1 Design Read Step Output

SpyGlass successfully elaborated `ASYNC_FIFO.v` and all sub-modules. The message tree shows **Total: 2 messages, Displayed: 2, Waived: 0** – both are informational Design Read messages (no errors or warnings).

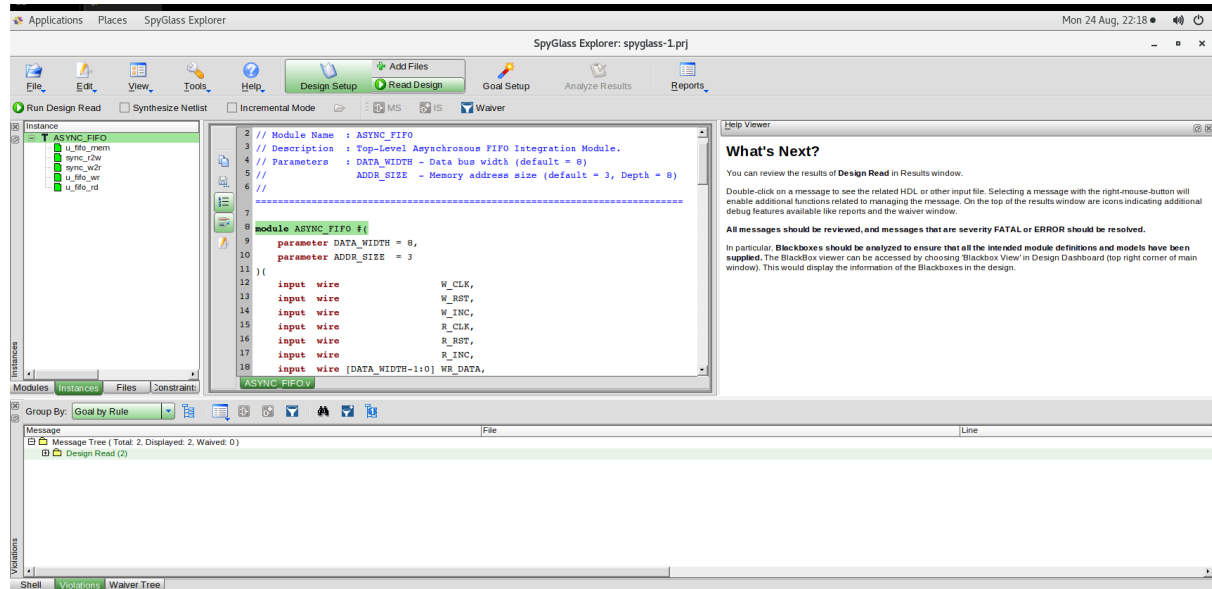


Figure 3: SpyGlass Design Read Step Output – 2 Info Messages, 0 Errors

6.2 Lint RTL Goal Run Output

The Lint RTL goal run completed with a **clean lint result** (0 Errors, 0 Warnings, 0 Latches).

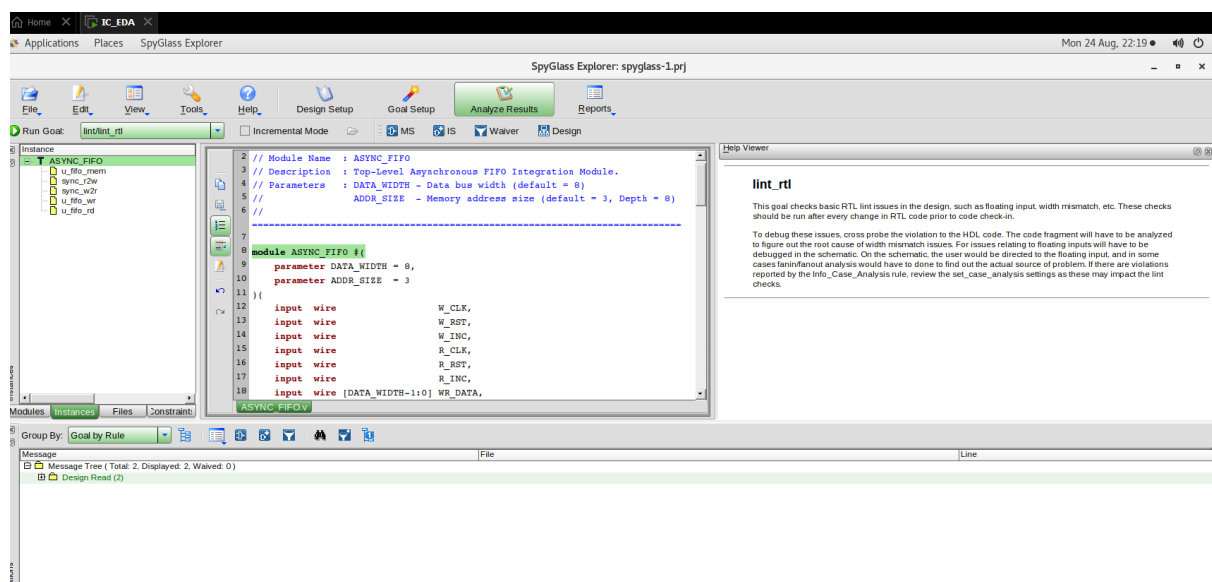


Figure 4: SpyGlass Lint RTL Goal Run Output – Clean Lint (0 Errors, 0 Warnings)

6.3 SpyGlass Lint Summary Report (moresimple.rpt)

```

1 Total Number of Generated Messages :      2
2 Number of Waived Messages          :      0
3 Number of Reported Messages        :      2
4 Number of Overlimit Messages       :      0
5
6 MORESIMPLE REPORT:
7
8 ##### BuiltIn -> RuleGroup=Design Read #####
9 [1] DetectTopDesignUnits  Info  ASYNC_FIFO.v  Line 8
10     Module ASYNC_FIFO is a top level design unit
11 [0] ElabSummary           Info
12     Please refer file elab_summary.rpt for elab summary report

```

Listing 7: SpyGlass moresimple.rpt – Lint Summary Report

7 Conclusion

The Asynchronous FIFO (ASYNC_FIFO) was successfully designed, modeled, and verified.

- Mathematical buffer depth calculation verified that an 8-entry FIFO (ADDR_SIZE = 3) fully accommodates a 9-byte burst write at 100 MHz with 40 MHz concurrent reading without overflow.
- Gray-code pointer encoding and double-flop synchronizers (DF_SYNC) eliminated CDC metastability.
- Registered full/empty flags eliminate combinational glitches and provide superior timing margin over the old un-registered implementation.
- All 4 simulation test cases passed with 0 errors in ModelSim.
- Clean SpyGlass lint report: 0 errors, 0 warnings, 0 latches.